

Erzeugung neuer Objekte:

```
Rechteck r = new Rechteck( );
```

Hierbei wird eine Konstruktor-Methode aufgerufen, die das neue Rechteck-Objekt erzeugt.

Konstruktor:

- Methode zur Erzeugung von Objekten
- Heißt genauso wie die Klasse
- Ziel: Setze die Attribute des neuen Objekts auf bestimmte Werte
- Vorgehen:
Zunächst erhalten die Attribute die Initialwerte aus den Variablen Deklarationen.
Anschließend wird der Code des Konstruktors ausgeführt.

```
class Rechteck {
    int laenge = 5;
```

```
    Rechteck() {
        "
    }
```

Bsp:

```
r = new Rechteck();
```

Dann hätte

" " " " " "

```

Rectangle ( ) {
    laenge ++;
}
:
}

```

überladen

v. laenge den Wert 6

- Es dürfen mehrere Methoden mit gleichen Namen (und daher auch mehrere Konstruktoren) auftreten, solange ihre Parameterlisten "hinreichend verschieden" sind.

⇒ Überladen von Methoden

Überladen

- Verschiedene Methoden dürfen den gleichen Namen haben, falls ihre Parameterlisten "verschieden" sind.
- Parameterlisten gelten als "verschieden", falls:
 - unterschiedliche Anzahl von Parametern oder
 - unterschiedliche Datentypen bei den Parametern
- Unterschiedliche Parameternamen oder unterschiedliche Resultattypen reicht nicht.

```

class A {
:
}

```

```

class A {
    :
    int f(int x) { ... }
    :
    double f(int y) { ... }
    :
}

```

← Versetzen

- Falls beim Aufruf einer überladenen Methode mehrere Implementierungen passen, dann wird die speziellste passende Methode ausgeführt.
- Überladung ist nur erlaubt, wenn die Auflösung beim Methodenaufruf eindeutig ist.

Bsp: Rechteck (double l, int b) { ... }

Rechteck (int l, double b) { ... }

Beim Aufruf von

new Rechteck (1, 2)

ist unklar, welcher Konstruktor ausgeführt werden soll

⇒ Fehler (Aufruf compiliert nicht)

Weiteres Bsp. für überladene Methode:

IO.print (...)

IO.print (...)

↑
gibt Argument auf Bildschirm aus

⇒ für verschiedene Argumenttypen vorhanden.

main ()

main (String[] args)

← Bei Ausführung der
Hauptmethode über

"java Date" wird

diese ausgeführt, falls beide
vorhanden sind.

Direkter Aufruf von
main() ist aber möglich.

Wenn man keinen Konstruktor schreibt, dann wird der
parameterlose Konstruktor (default constructor) auto-
matisch erzeugt.

Achtung:

1) Sobald man einen Konstruktor implementiert
(ggf. mit Parametern), dann wird in der Klasse
kein parameterloser Konstruktor mehr erzeugt.

2) Bei Aufruf von nicht-statischem main (über
"java Date") wird ein Objekt über den
parameterlosen Konstruktor erzeugt. Den muss es

parameterlosen Konstruktor erzeugt. Den muss es dann auch geben, wenn dieses main User "java Date" aufgerufen wird.

Zugriff auf das "aktuelle" Objekt: this

d.h. auf das gerade erzeugte Objekt beim Konstruktor bzw. auf das Objekt, für das gerade eine nicht-statische Eigenschaft berechnet wird.

```
class Rechteck {  
    :  
    double flaeche() {  
        return this.laenge * this.breite;  
    }  
    :  
}
```

Aufruf mit  `r.flaeche()`.
↙ aktuelle Objekt

Wenn man nicht-statische Eigenschaften ohne `.` aufruft, dann wird automatisch (implizit) `"this."` ergänzt.

⇒ Führt in statischen Methoden zu Fehlern,

d.h. dass ist `r.flaeche()` erlaubt,
aber nicht `this.flaeche()` oder `flaeche()`.

Weiterer typischer Konstruktor: Kopier-Konstruktor

Records

- Ziel: spare Schreibarbeit bei "Daten-Klassen", bei denen Attribute unveränderlich sein sollen.
- Records sind spezielle Klassen mit eigener Syntax
- Attribute direkt im Kopf deklariert
- Attribute sind automatisch private + final.
- keine weiteren nicht-statischen Attribute erlaubt.
Aber static Attribute und static + non-static Methoden sind erlaubt.
- Es wird automatisch ein "canonical constructor" erzeugt, der die Werte aller Attribute setzt (aber kein parameterloser Konstruktor)
- Es werden auch automatisch "getter"-Selektoren und toString-Methode erzeugt.
- Man kann weitere Konstruktor schreiben u. auch canonical constructor und toString selbst schreiben.
- Weitere Möglichkeit: "Compact Constructor":
Ergänzt den canonical constructor um Code, der

Ergänzt den canonical constructor um Code, der am Anfang des Konstruktors ausgeführt wird.